

Análisis de Algoritmos I

Algoritmos de ordenamiento

María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Algoritmos de ordenamiento

- **Objetivo:** Poner elementos de una lista en una secuencia dada por una **relación de orden**, es decir, la salida ha de ser una permutación (o reordenamiento) de la entrada que satisfaga la relación de orden dada.
- Las relaciones más comunes son: el orden **numérico** y el orden **lexográfico**, ya sea ascendente o descendente.
- Ordenar de forma eficiente es importante para optimizar la eficiencia de otros algoritmos que requieren de listas ordenadas (como los de búsqueda y fusión).

Algoritmos de ordenamiento

- **Antes de las computadoras:** ordenar a mano

- Ordenar cartas
- Archivos en papel
- Listas telefónicas
- Diccionesarios
- ...

- De ahí surgen ideas naturales como:

- comparar vecinos
- insertar en el lugar correcto

- **Años 50:** con las primeras computadoras nacen los primeros algoritmos “formales”

- ¿Cómo ordenamos datos que sea fácil de entender y fácil de programar?



Insertion sort



Bubble sort

Algoritmos de ordenamiento

- **Años 60:** la eficiencia importa
 - Las computadoras empiezan a manejar más datos y ordenar mal es costoso.
 - Uno de los primeros algoritmos eficientes, diseñado por **John von Neumann**, pensando en las cintas magnéticas y memoria limitada.
 - Divide el problema en partes iguales.
- **Años 60 – 70:** rapidez práctica
 - Algoritmo simple pero muy rápido en la práctica
 - Desarrollado por **Tony Hoare**, estudiante de universidad



Merge sort



Quick sort

Algoritmos de ordenamiento

- **Años 70 – 80:** se establece un límite teórico
 - No existe un algoritmo de ordenamiento por comparación mejor que $O(n \log n)$
 - El objetivo ya no es ser más rápidos, sino:
 - usar menos memoria
 - ser estable
 - aprovechar el orden previo
 - adaptarse al hardware
- **Años 90 – Presente:** algoritmos híbridos
 - Los lenguajes actuales no usan un solo algoritmo.
 - Python: Timsort → mezcla insertion + merge
 - Java / C++: quick sort + heap sort



Heap sort
Intro sort



Tim sort
Radix sort
Count sort

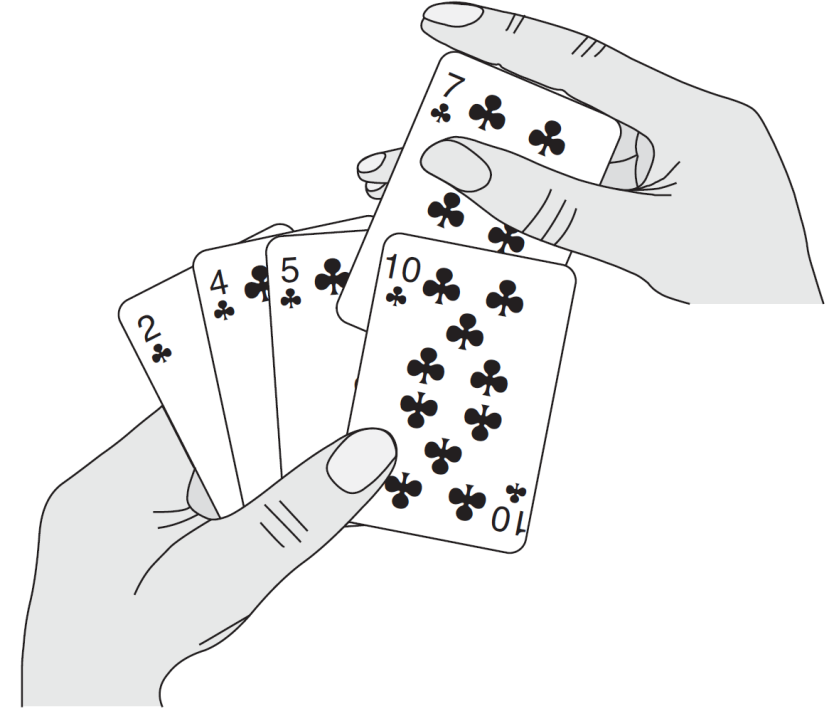
Algoritmos de ordenamiento

- Los números que queremos ordenar se les llama **llaves (keys o sort keys)**.
- Usualmente vienen acompañados de otros datos, información asociada a las llaves, a la cual se le llama **datos satélite (satellite data)** que deben moverse junto con la llave.
 - Ejemplo: cuando ordenamos una columna de una tabla en Excel y te pregunta si se va extender el ordenamiento al resto de las columnas.
- Visualizaciones:
 - <https://visualgo.net/en/sorting>
 - <https://es.khanacademy.org/computing/computer-science/algorithms/insertion-sort/a/insertion-sort-pseudocode>

Insertion sort

(Ordenamiento por inserción)

- Analiza de izquierda a derecha, uno por uno, los datos de una sucesión.
- Una vez que se ha analizado cada dato, se inserta en un sitio idóneo en una sucesión ya ordenada.
- Es eficiente para listas pequeñas o cuando la lista está casi ordenada.



Insertion sort: Algoritmo

11, 7, 14, 1, 5, 9, 10.



Sucesión ordenada

11

7, 11

7, 11, 14

1, 7, 11, 14

1, 5, 7, 11, 14

1, 5, 7, 9, 11, 14

1, 5, 7, 9, 10, 11, 14

Sucesión no ordenada

7, 14, 1, 5, 9, 10

14, 1, 5, 9, 10

1, 5, 9, 10

5, 9, 10

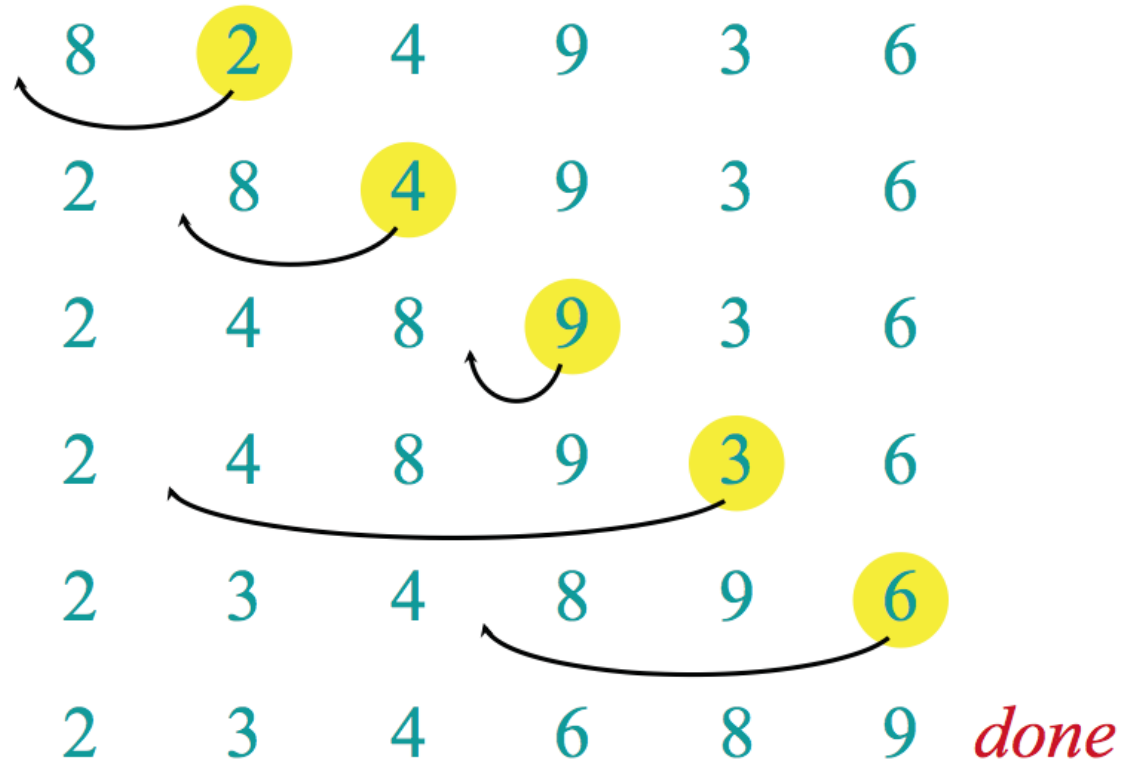
9, 10

10

Ejemplo
Insertion
Sort

Insertion sort: Ejemplo

$A=[8,2,4,9,3,6]$



Ejemplo
Insertion
Sort

Insertion sort: pseudocódigo

Algoritmo #3

Input: $x_1, x_2, \dots, x_n$

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

for $j = 1$ to n **do**

$i = j - 1$ #izq

$x = x_j$ #der #n-1 veces

while $x < x_i$ and $i \geq 0$ **do**

$x_{i+1} = x_i$ #despl. a la der p/ abrir espacio #por c/iter j: d_i

$i = i - 1$

$x_{i+1} = x$ #insertar a la izq #n-1 veces

Insertion sort: Análisis

En este análisis, tomaremos el **número de intercambios de datos** como lo que va a medir la complejidad temporal del algoritmo:

$$\textbf{(a)} \ x = x_j \quad \textbf{(b)} \ x_{i+1} = x_i \quad \textbf{(c)} \ x_{i+1} = x$$

Tenemos dos ciclos: uno exterior (**for**) y otro interior (**while**). En el ciclo exterior siempre se ejecutan dos operaciones de intercambio de datos: (a) y (c). Debido a que el ciclo interior puede o no ser ejecutado, el número de intercambios de datos realizados para x_j en el ciclo interior se denotará por d_j .

El número total de movimientos de datos para el ordenamiento por inserción directa es:

$$\begin{aligned} X &= \sum_{i=2}^n (2 + d_i) \\ &= 2(n-1) + \sum_{i=2}^n d_i \end{aligned}$$

Insertion sort: Mejor caso

- Cuando los datos de entrada **ya están ordenados**.
- El algoritmo nunca ejecuta el ciclo interior *while*.
- Únicamente se recorren las n posiciones del arreglo para confirmar que la información de cada casilla del arreglo está en la posición donde tiene que quedar.
- Número de intercambios:

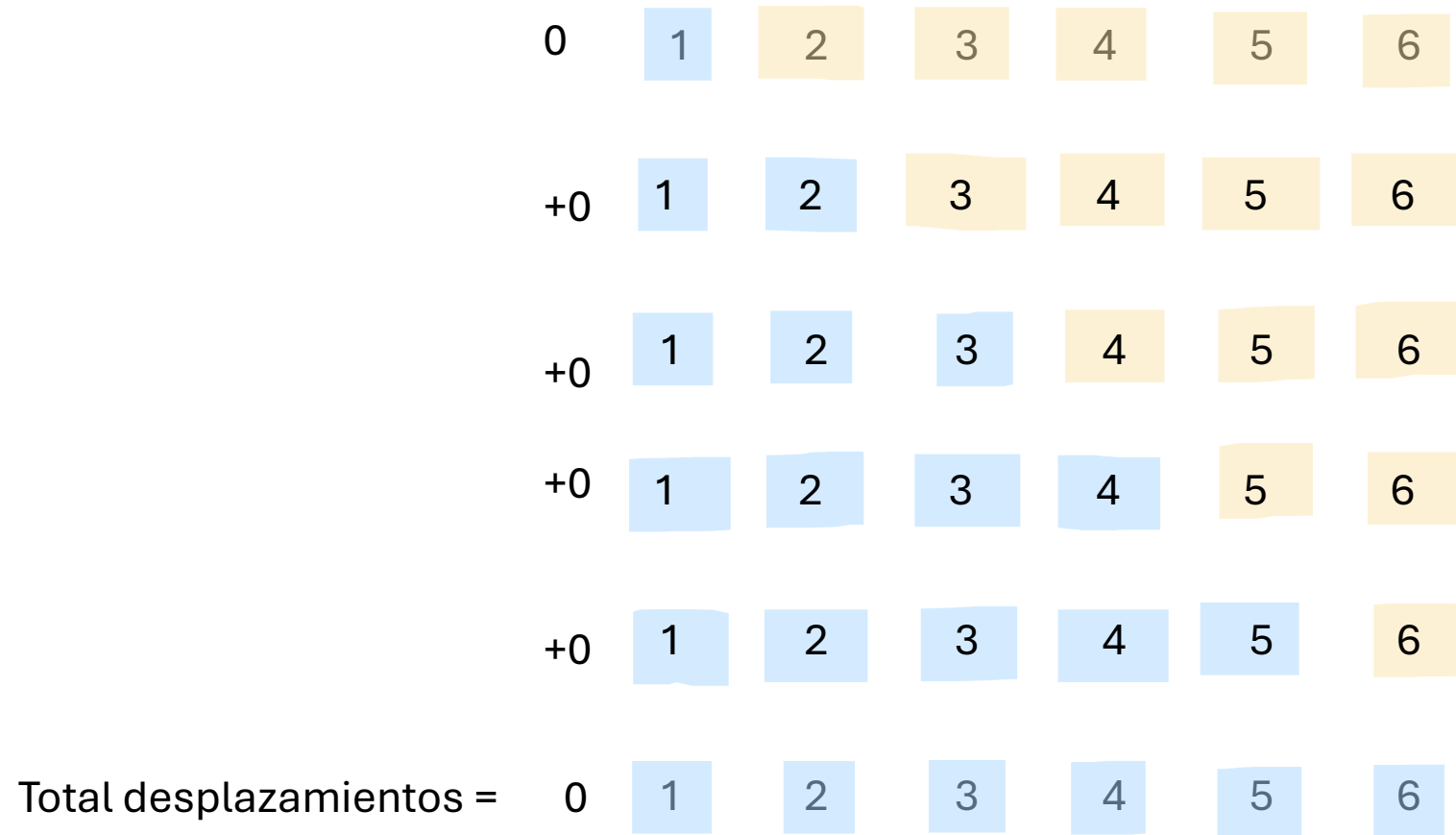
$$\sum_{i=2}^n d_i = 0,$$

$$A = [1, 2, 3, 4, 5, 6]$$

$$X = 2(n - 1)$$

- **Complejidad:** $O(n)$

Insertion sort: Mejor caso



En el mejor caso para $n=6$, el total sería 0 desplazamientos; es decir, se realizan $n-1$ comparaciones y ningún desplazamiento.

Insertion sort: Peor caso

- Cuando los datos de entrada están **ordenados a la inversa**.
- El ciclo *while* se ejecuta el **número máximo posible de iteraciones**: una vez para el elemento de la posición dos, se ejecuta dos veces para el elemento de la posición tres, se ejecuta tres veces para el elemento de la posición cuatro, y así sucesivamente hasta llegar a la posición n donde el ciclo se tiene que ejecutar $n-1$ veces.
- Número de intercambios:

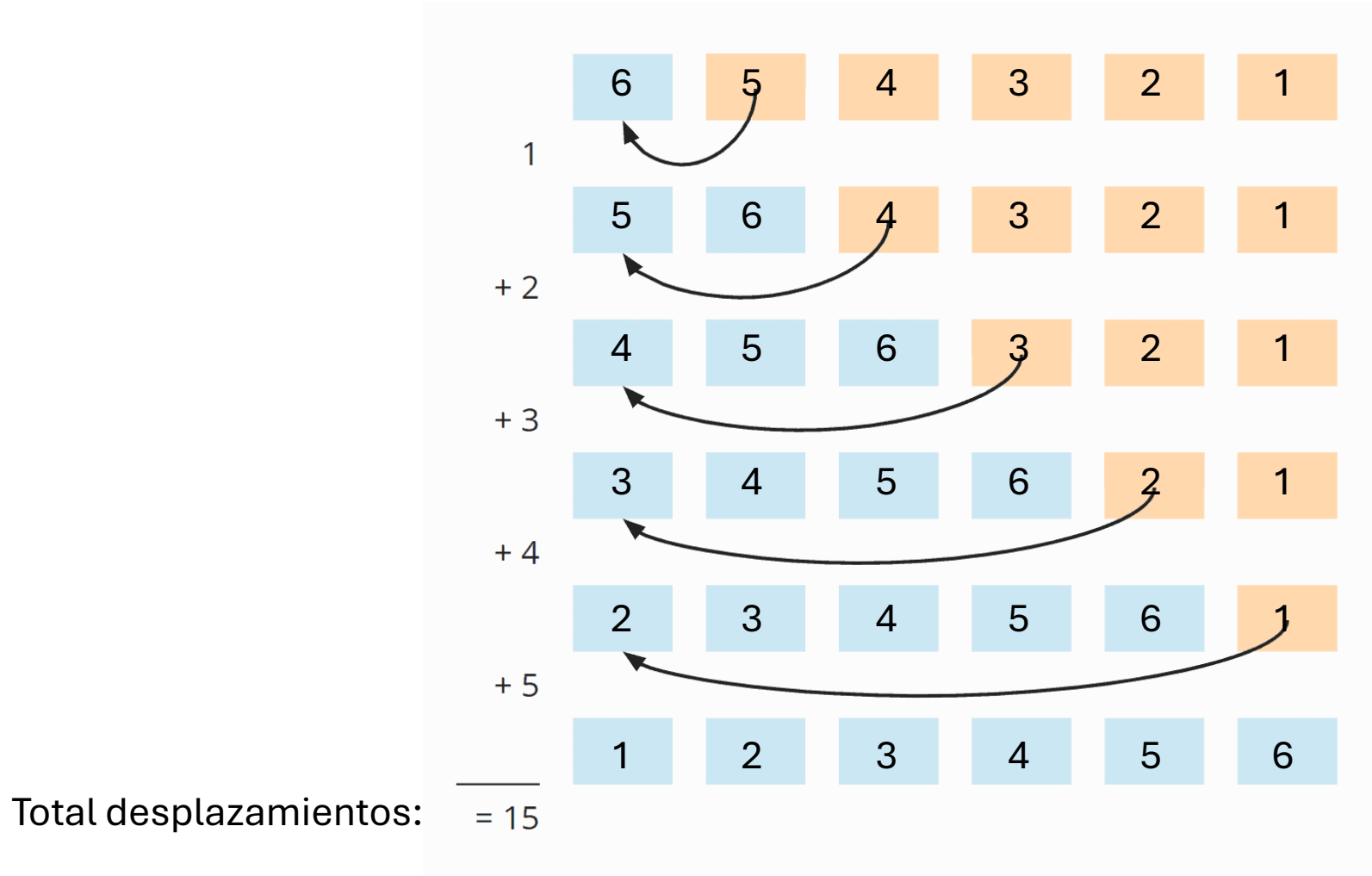
$A = [6, 5, 4, 3, 2, 1]$

Iteración (j)	Desplazamientos (d_j)	
2	1	$d_2 = 1$
3	2	$d_3 = 2$
4	3	$\vdots$
...	...	$\vdots$
n	n-1	$d_n = n - 1.$

$$= \frac{n(n-1)}{2} \approx \frac{n^2}{2}$$

- Si tenemos n elementos y cada uno se compara con una cantidad de elementos que depende de n , terminas haciendo aproximadamente **$n \times n$ operaciones**.
- **Complejidad:** $O(n^2)$

Insertion sort: Peor caso



Cada elemento nuevo tiene que compararse con prácticamente todos los que ya estaban ordenados.

Coincide con la lógica de que el número de desplazamientos $= \frac{n(n-1)}{2} = \frac{6(6-1)}{2} = \frac{6(5)}{2} = \frac{30}{2} = 15$.

Insertion sort: Caso promedio

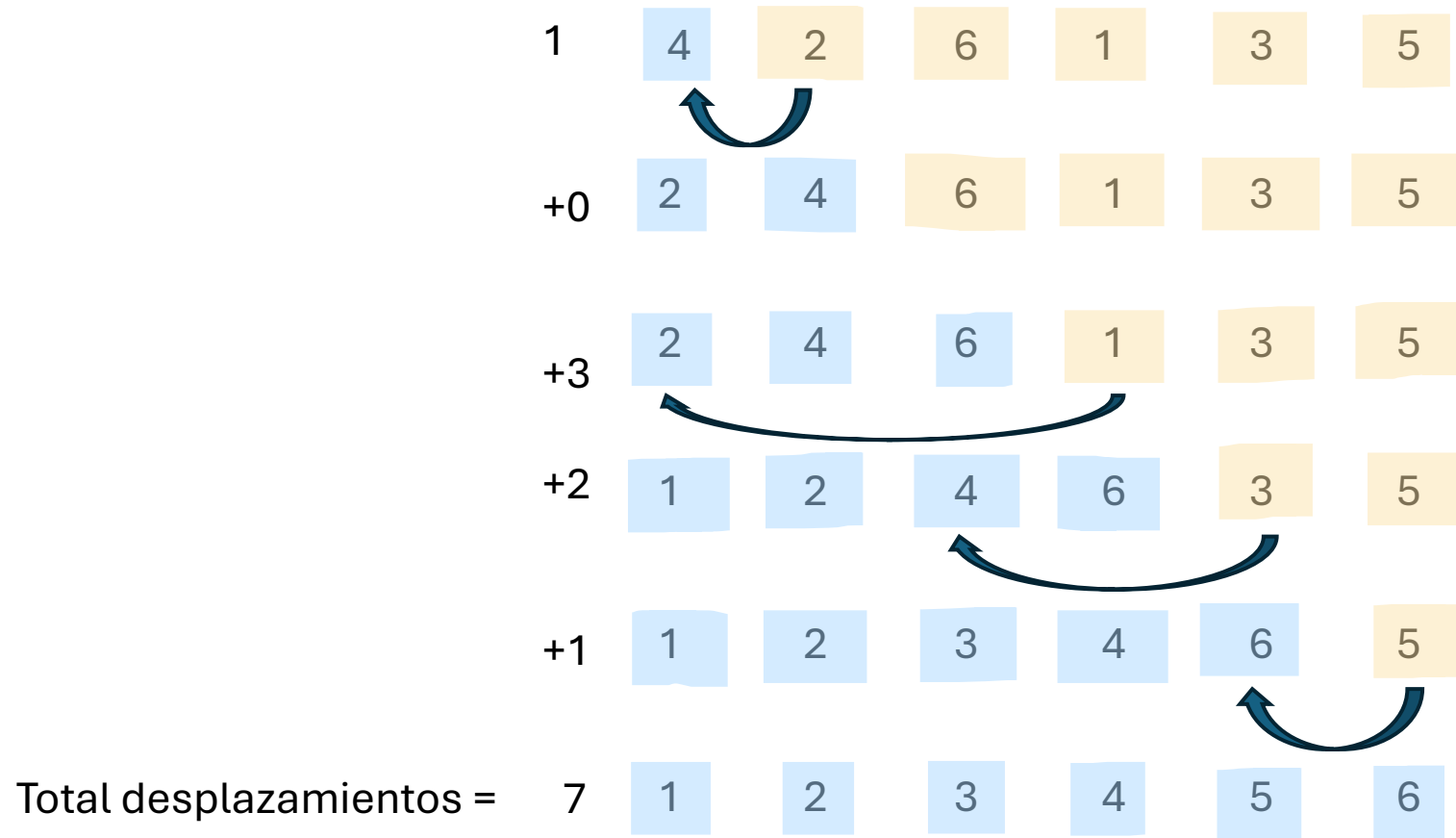
- Cuando los datos de entrada están en **orden aleatorio**, y en promedio, cada elemento termina moviéndose **la mitad de lo que podría moverse**.
- El ciclo interior se ejecutará aproximadamente **la mitad** de comparaciones.

- Número de comparaciones: $\approx \frac{n^2}{4}$

A = [4, 2, 6, 1, 3, 5]

- En promedio, haces **la mitad del trabajo que en el peor caso**, pero como ese trabajo sigue dependiendo de cuántos elementos ya ordenaste, el crecimiento sigue siendo cuadrático.
- **Complejidad:** $O(n^2)$

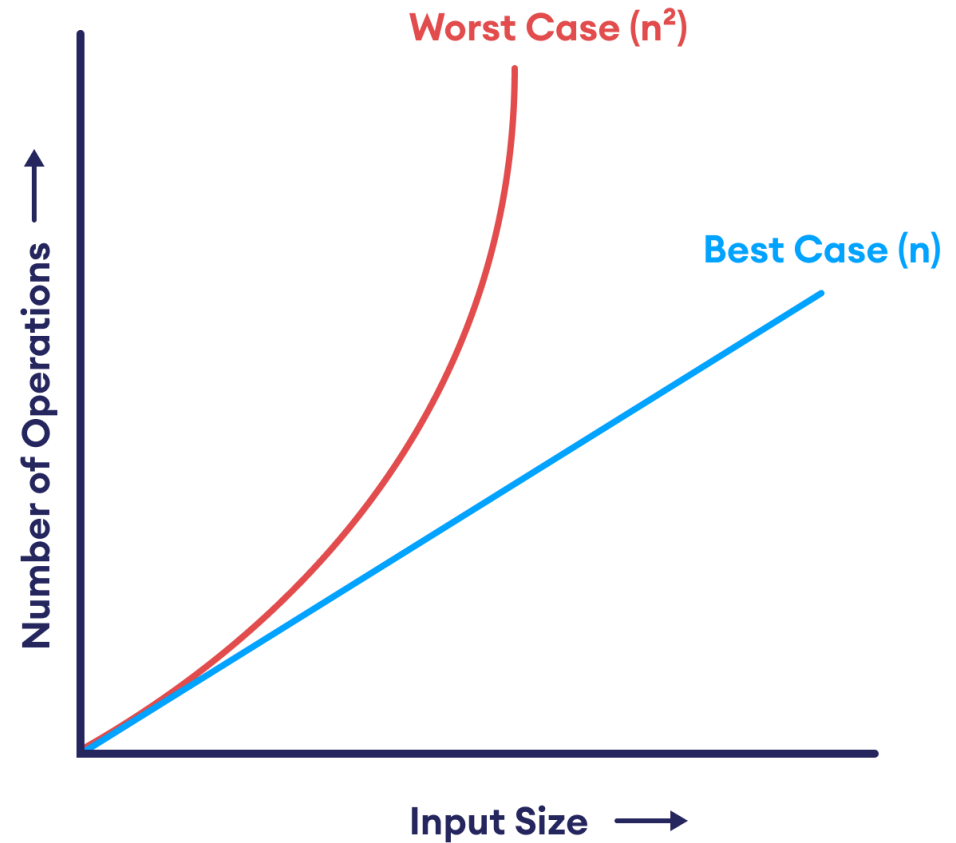
Insertion sort: Caso promedio



En el peor caso para $n=6$, el total sería 15 desplazamientos; aquí estamos cerca de la mitad, cumpliendo la lógica del promedio.

Insertion sort: Resumen

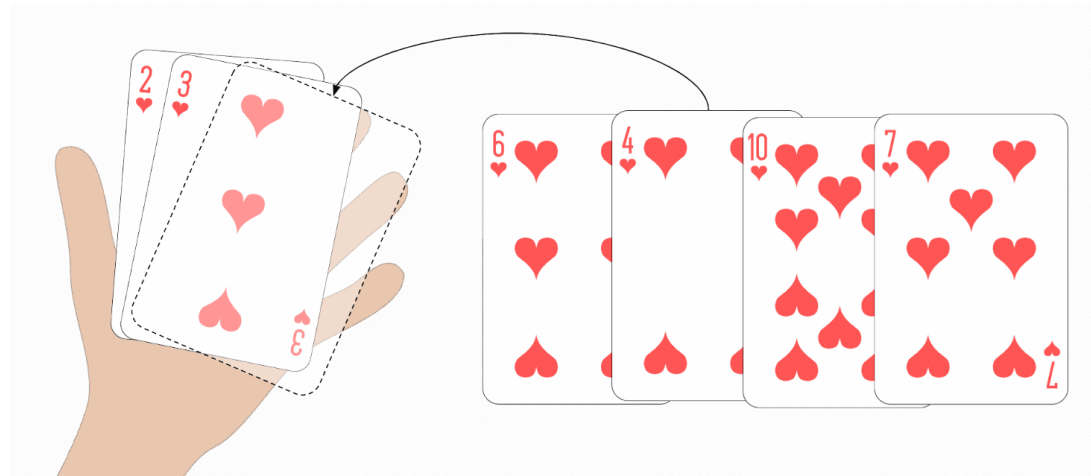
- **Mejor caso:** $O(n)$
- **Peor caso:** $O(n^2)$
- **Caso promedio** $O(n^2)$



Selection sort

(Ordenamiento por selección directa)

- Se encuentra el número más pequeño de un arreglo A , y si es necesario, se intercambia de tal manera que ocupe la posición a_1 del arreglo.
- Se repite el paso anterior, en busca del segundo número más pequeño y se coloca en la posición a_2 .
- El proceso continúa hasta que se encuentra el número más grande.
- Puede ser útil en listas pequeñas o cuando hay un entorno de memoria limitado. Pero es muy ineficiente con grandes cantidades de datos.



Selection sort: pseudocódigo

Algoritmo #4

Input: $x_1, x_2, \dots, x_n$

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

```
for  $i = 1$  to  $n-1$  do                                #n-1 veces
    pos_min = i                                         #pivote para indicar el mínimo

    for  $j = i+1$  to  $n$  do                                #n-i veces en cada iteración
        if  $x[j] < x[pos\_min]$  then
            pos_min = j                                #se actualiza la posición min
    temp = x[j]                                         #se intercambia
    x[j] = x[pos_min]
    x[pos_min] = temp
```

Selection sort: Análisis

Tenemos dos operaciones importantes: (a) la comparación de dos elementos y (b) el cambio del pivote:

$$\textbf{(a)} \ x[j] < x[\text{pos_min}] \qquad \textbf{(b)} \ \text{pos_min} = j$$

Ambas operaciones tendrán un tiempo de ejecución constante $O(1)$. Por lo tanto, la clave para analizar este algoritmo está en contar el **número de iteraciones que realiza el bucle interno**:

- Cuando $i = 1$, el bucle interno j va desde 2 hasta n , es decir, hace $n-1$ iteraciones.
- Cuando $i = 2$, el bucle interno j va desde 3 hasta n , es decir, hace $n-2$ iteraciones.
- Cuando $i = 3$, el bucle interno j va desde 4 hasta n , es decir, hace $n-3$ iteraciones.
- En general, el bucle interno corre $n-i$ veces.
- En la última vuelta cuando $i = n-1$, el bucle interno solo realiza 1 iteración.

Selection sort: Análisis

Por lo tanto, el número de iteraciones del bucle interno se puede expresar como la siguiente serie:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

A esta sumatoria se le conoce como **serie aritmética** y se puede definir como:

$$\sum_{i=1}^k i = \frac{k (k + 1)}{2}$$

Sustituyendo k por $(n - 1)$, tenemos que el total de iteraciones será:

$$\frac{(n - 1)((n - 1) + 1)}{2} = \frac{(n - 1)(n)}{2} = \frac{n^2 - n}{2}$$

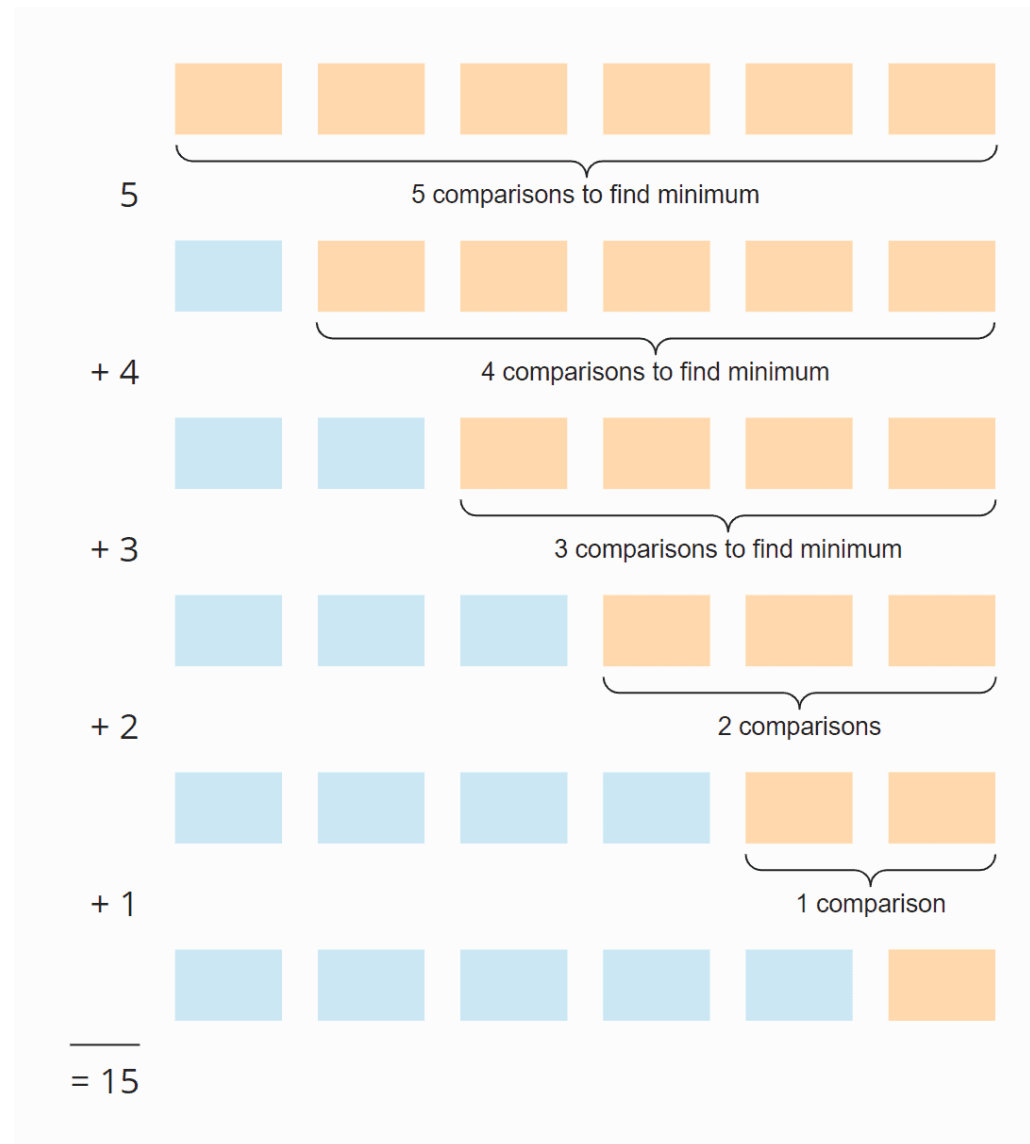
Simplificando, tenemos que la complejidad para este algoritmo será de **$O(n^2)$** .

Selection sort: Todos los casos

- Esto cubre todos los casos, ya que independientemente de si el arreglo ya está ordenado o no, el bucle interno siempre realizará las mismas comparaciones, por lo que siempre será **$O(n^2)$** .
- Esto se debe a que los bucles *for* están definidos por el tamaño n y no dependen de una condición de salida temprana (como sí ocurre en el Insertion Sort).

Selection sort: Todos los casos

- **Mejor caso:** el arreglo ya está ordenado.
 - Se realizan todas las comparaciones $O(n^2)$, pero el pivote $pos_min = j$ nunca cambia dentro del *if*.
- **Peor caso:** el arreglo está en orden inverso.
 - Se realizan todas las comparaciones $O(n^2)$ y el pivote $pos_min = j$ cambia en cada paso posible.
- **Caso promedio:** datos aleatorios.
 - Se realizan todas las comparaciones $O(n^2)$ y el pivote cambia de forma intermitente.

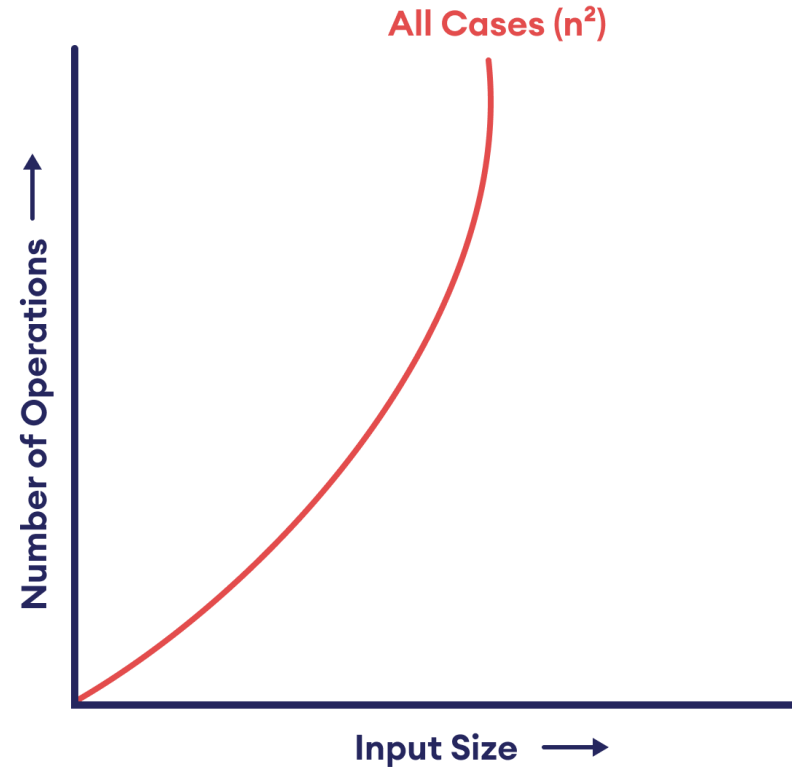


Dado que el número de comparaciones que se harán siempre será el mismo para todos los casos podemos calcularlo como: $\frac{n^2 - n}{2}$, por lo tanto para $n = 6$: $\frac{6^2 - 6}{2} = 30/2 = 15$ comparaciones

Ejemplo Selection sort

Selection sort: Resumen del análisis

- **Mejor caso:** $O(n^2)$.
- **Peor caso:** $O(n^2)$.
- **Caso promedio:** $O(n^2)$.



Bibliografía

- Lee, R.C.T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2007). Introducción al diseño y análisis de algoritmos. Un enfoque estratégico. México, D.F. Mc Graw Hill.
- Cormen T. H., Leiserson C. E., R. L. Rivest y C. S. (2022). *Introduction to Algorithms* .
- Cormen, T. H. (2013). *Algorithms Unlocked*.
- https://en.wikipedia.org/wiki/Sorting_algorithm
- https://es.wikipedia.org/wiki/Ordenamiento_por_inserci%C3%B3n
- <https://visualgo.net/en/sorting>
- <https://dipeshpatil.github.io/algorithms-visualiser/#/>
- https://es.wikipedia.org/wiki/Ordenamiento_de_burbuja
- <https://www.bigocheatsheet.com/>
- https://es.wikipedia.org/wiki/Ordenamiento_por_mezcla
- https://developer-nvidia-com.translate.goog/blog/merge-sort-explained-a-data-scientists-algorithm-guide/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge#:~:text=Para%20implementarlo%20usted%20mismo:,una%20lista%20combinada%20y%20ordenada.